Despliegue gratis en Hugging Face Spaces

Proyecto Café ML2 · Qué es, por qué funciona y cómo hacerlo paso a paso

En una frase: Hugging Face Spaces toma nuestro **Dockerfile**, construye la imagen, ejecuta el contenedor en sus servidores y nos da una **URL pública** para la app. Es gratis y **no pide tarjeta**. Toma unos 15 minutos.

Repositorio del proyecto: github.com/danielaestudio02-del/Despliegue_cafe_ml2

1. ¿Qué es Hugging Face?

Hugging Face es una plataforma muy usada en Machine Learning. Es como un "GitHub para modelos de inteligencia artificial": la gente comparte ahí modelos, datos y aplicaciones de demostración.

Un **Space** es un espacio dentro de Hugging Face para **publicar una aplicación**. Uno sube los archivos del proyecto, Hugging Face lo ejecuta en sus computadores y le da una dirección web para que cualquiera la use. El plan básico (**CPU basic**) es gratuito.

Los Spaces pueden funcionar de varias formas. Nosotros usamos la opción **Docker**, que es justamente lo que pide el taller: que la app se ejecute dentro de un contenedor construido con nuestro Dockerfile.

2. ¿Por qué lo usamos en vez de Google Cloud?

- La guía del profe pide una máquina virtual (VM) de Google Cloud, pero para activarla Google pidió **tarjeta y un prepago**, y no contamos con ese dinero.
- Hugging Face Spaces es **gratis, sin tarjeta**, y usa **Docker de verdad**: construye la imagen con el mismo Dockerfile que usaríamos en la VM.
- Así cumplimos la parte de **Docker** y tenemos una **URL pública** para la demostración y la prueba cruzada con otra pareja.

Ojo con la nota: esto **no es Google Cloud**. En la rúbrica, GCP vale 15%. Hay que confirmar con el profe si acepta Hugging Face como alternativa, o si tiene créditos de Google Cloud for Education (que no piden tarjeta).

3. ¿Por qué funciona? Lo que pasa por dentro

Es **el mismo proceso del taller** (Dockerfile → imagen → contenedor), solo que Hugging Face hace automáticamente los pasos que en la VM haríamos a mano:

En la VM de Google Cloud (a mano)	En Hugging Face Spaces (automático)
Crear la máquina virtual	Hugging Face pone el computador
Instalar Docker y Git	Ya están instalados
git clone del repositorio	Subimos los archivos al Space
docker build → crea la imagen	Lo hace solo al subir los archivos (estado **Building**)
docker run -p 8501:8501 → crea el contenedor	Lo hace solo (estado **Running**)
Regla de firewall para el puerto 8501	La línea **app_port: 8501** del README
Abrir **http://IP_EXTERNA:8501**	Abrir la URL del Space (con https)

¿Cómo sabe Hugging Face qué hacer?

Por las primeras líneas del **README.md** del Space. Ese bloque entre las líneas **---** es la configuración:

```
---
title: Cafe ML2 Segmentacion
emoji: ☕
colorFrom: yellow
colorTo: green
sdk: docker        <- "construye la app con el Dockerfile"
app_port: 8501     <- "la app escucha en el puerto 8501"
pinned: false
---
```

- **sdk: docker** le dice que busque el **Dockerfile** y lo use para construir la imagen.
- **app_port: 8501** le dice a qué puerto del contenedor debe enviar a los visitantes. Streamlit escucha en el 8501 porque así lo indica el **CMD** del Dockerfile. Si estos dos números no coinciden, la página se queda cargando.

¿Por qué la app da los mismos resultados que en el notebook?

- **model.joblib** es el modelo ya entrenado en Colab. La app solo lo carga; no vuelve a entrenar.
- **requirements.txt** fija las **mismas versiones** de librerías con las que se entrenó el modelo (por ejemplo **scikit-learn==1.6.1**). Si Docker instalara otras versiones, el modelo podría no cargar. Por eso las fijamos.
- Lo probamos antes: se construyó la imagen con los archivos de GitHub, se ejecutó como lo hace Hugging Face (usuario sin permisos de administrador) y los 3 casos de prueba dieron el cluster esperado.

Qué hace cada archivo que vamos a subir

Archivo	Para qué sirve
Dockerfile	La "receta": parte de Python 3.12, instala las librerías y arranca Streamlit en el puerto 8501.
requirements.txt	La lista de librerías con sus versiones exactas.
app.py	La aplicación de Streamlit: el formulario y el resultado del cluster.
model.joblib	El modelo K-Means entrenado, con las variables y la descripción de cada cluster.
README.md (de la carpeta huggingface_space)	La configuración del Space: usar Docker y el puerto 8501.

4. Paso a paso

Paso 1. Crear la cuenta

1. Entrar a **huggingface.co/join**.
2. Registrarse con cualquier correo (sirve el institucional). No pide tarjeta.
3. Confirmar el correo con el enlace que llega.

Paso 2. Crear el Space

Entrar a **huggingface.co/new-space** y llenar así:

Campo	Valor
Space name	cafe-ml2
License	mit (opcional)
Select the Space SDK	**Docker**, plantilla **Blank**
Space hardware	**CPU basic · Free**
Visibility	**Public**

Presionar **Create Space**.

Paso 3. Descargar los 5 archivos de GitHub

En **github.com/danielaestudio02-del/Despliegue_cafe_ml2**, abrir cada archivo y presionar el botón **Download raw file** (ícono de descarga, arriba a la derecha):

1. **Dockerfile**
2. **app.py**
3. **model.joblib**
4. **requirements.txt**
5. **huggingface_space/README.md**: entrar a la carpeta **huggingface_space** y descargar **ese** README, no el de la raíz.

Revisar que en el computador queden con esos nombres exactos (sin "(1)" ni extensiones extra).

Paso 4. Subir los archivos al Space

1. En el Space, ir a la pestaña **Files**.
2. Presionar **+ Contribute → Upload files**.
3. Arrastrar los 5 archivos juntos.
4. Presionar **Commit changes to main**.

> El README que subimos **reemplaza** al que Hugging Face crea solo, y está bien: sus primeras líneas (**sdk: docker** y **app_port: 8501**) son las que hacen funcionar el Space. No borrarlas.

Paso 5. Esperar a que se construya

1. Ir a la pestaña **App**. Arriba aparece **Building** mientras construye la imagen (3 a 5 minutos).
2. Con el botón **Logs** se ve el **docker build** en vivo. Tomar una captura: es evidencia de que se usa Docker.
3. Cuando cambie a **Running**, aparece la app.

Paso 6. Probar y guardar evidencia

Probar los 3 casos y tomar capturas:

Cantidad	Precio unitario	Gasto total	Debe dar
1	2	2	**Cluster 2**
5	1	5	**Cluster 1**
4	5	20	**Cluster 0**

URL pública para compartir con el profe y la otra pareja:

- **https://huggingface.co/spaces/USUARIO/cafe-ml2**
- o directa: **https://USUARIO-cafe-ml2.hf.space**

Paso 7. Completar el README del repositorio

- En la sección **“Despliegue (Parte 2)”**: explicar que se usó Hugging Face Spaces con Docker, la URL del Space y las capturas.
- En **“Autores”**: los nombres de las dos.

5. A tener en cuenta

- **Se duerme sola:** si nadie la usa por un tiempo, la app se pausa. Al abrirla despierta en 1 o 2 minutos. Abrirla un rato antes de la demostración.
- **Para actualizar:** subir de nuevo el archivo cambiado; el Space se reconstruye solo.
- **Es pública:** cualquiera con el enlace la puede ver. No subir datos personales ni claves.

Si algo falla

Problema	Solución
Sale **Configuration error**	El README subido no es el de la carpeta **huggingface_space**, o se borraron sus primeras líneas.
Se queda en **Build error**	Abrir **Logs**: ahí aparece el error exacto. Revisar que estén los 5 archivos con su nombre correcto.
La página se queda cargando	Revisar que el README diga **app_port: 8501**.
Error al cargar model.joblib	Volver a descargar **model.joblib** de GitHub con **Download raw file** y subirlo de nuevo.
Dice **Sleeping**	Es normal: presionar **Restart this Space** y esperar.

Checklist final

- ☐ La cuenta de Hugging Face está creada.
- ☐ El Space **cafe-ml2** está creado con SDK **Docker** y hardware gratuito.
- ☐ Se subieron los 5 archivos, con el README de la carpeta **huggingface_space**.
- ☐ El estado pasó de **Building** a **Running**.
- ☐ Los 3 casos de prueba dan el cluster esperado.
- ☐ Tenemos capturas de los **Logs** (Docker) y de la app funcionando.
- ☐ El README del repositorio tiene la URL, la sección de despliegue y los autores.
- ☐ Le preguntamos al profe si acepta Hugging Face como alternativa a GCP.